

Algorithm and Architecture Co-Design of Hardware-Oriented, Modified Diamond Search for Fast Motion Estimation in H.264/AVC

Obianuju Ndili and Tokunbo Ogunfunmi, *Senior Member, IEEE*

Abstract—In this paper, we present a new hardware-oriented, modified diamond search (HMDS) algorithm, for fast integer pel, motion estimation in H.264/AVC. We also present our co-designed, low power very large scale integration (VLSI) architecture for HMDS. The goal of HMDS is to enable the support of high quality video on low power mobile devices and low bit rate applications which typically use H.264/AVC baseline profile at levels 1–2. Our experiments use standard test sequences ranging from QCIF to high-definition 1280 × 720p video. The proposed VLSI architecture is prototyped as an field-programmable gate array (FPGA)-based field programmable system-on-chip. Our results show that HMDS on average has better rate-distortion performance and speedup, compared to previous state-of-the-art fast motion estimation algorithms, while its losses compared to full search motion estimation, are insignificant. Our prototyped architecture is more hardware-efficient than previous FPGA-based architectures in terms of power consumption, area, throughput, and memory utilization. We also show that its performance in terms of maximum frequency, minimum frequency, transistor count, and power consumption are comparable to that of state-of-the-art architectures implemented on application-specific integrated circuits.

Index Terms—Fast integer motion estimation, FPGA, H.264/AVC, VLSI architecture.

I. INTRODUCTION

THERE IS AN increasing need for high quality, real-time video on low power, mobile devices such as mobile phone cameras, bio-medical devices, security and sensor cameras, and so on. These devices often operate in low bit rate scenarios such as mobile and wireless communication services.

Therefore they typically target the support of H.264/AVC baseline profile at levels 1–2. This in turn means support of I and P slices, bit rates up to 2 Mb/s, frame resolutions and rates up to SIF/CIF at 30 Hz, 5–6 reference frames, CAVLC entropy coding, 4:2:0 chroma format, and 8-bit sample depth. In a few of these low power devices, the target includes supporting B slices and frame resolutions and rates up to high-definition 720p@60 Hz.

Manuscript received June 9, 2010; revised November 6, 2010; accepted January 21, 2011. Date of publication March 28, 2011; date of current version September 2, 2011. This paper was recommended by Associate Editor W. Gao.

The authors are with the Department of Electrical Engineering, Santa Clara University, Santa Clara, CA 95053 USA (e-mail: ondili@scu.edu; togunfunmi@scu.edu).

Color versions of one or more of the figures in this paper are available online at <http://ieeexplore.ieee.org>.

Digital Object Identifier 10.1109/TCSVT.2011.2133990

Motion estimation (ME) is the most powerful compression tool in the H.264/AVC [1], but it is also the most time consuming. In order to meet low power and real-time constraints, it is critical to efficiently speed up ME via fast algorithms and hardware acceleration.

A number of fast ME algorithms have been proposed in the literature. These include algorithms with a fixed set of search patterns such as three step search (TSS) [2], new three step search [3], four step search (4SS) [4], diamond search (DS) [5], [6], and hexagon-based search [7]. These algorithms suffer from the “local minima problem” because they have less adaptability and search efficiency to track large motions, and they are based on the assumption that the error surface increases monotonically as the search position moves away from the global minimum. An example of such an error surface is shown in Fig. 1(a). While this assumption may be true in a small region surrounding the global minimum (a radius of about 2 pixels), it is not true in general for video signals. This is illustrated by the error surface shown in Fig. 1(b).

However the afore-mentioned algorithms have the important advantage of simplicity and regularity, which in turn makes them attractive for hardware implementation. Among them, DS was accepted for MPEG-4 verification model [8].

In order to better track large motions, some advanced algorithms have been proposed. These include adaptive algorithms with adaptive search patterns and more recently, hybrid algorithms which combine various search patterns along with adaptive techniques, to avoid being trapped in local minima. Well-known examples of adaptive algorithms include enhanced predictive zonal search (EPZS) [9]. Well-known examples of hybrid algorithms include the cross-diamond search [10], cross-diamond-hexagonal search [11], unified multi-hexagon search UMHExagonS [12], and the simplified unified multi-hexagon (SUMH) search [13]. EPZS, UMHExagonS, and SUMH are non-normative parts of the H.264/AVC standard.

A major drawback of advanced fast ME algorithms is that they are not suitable for hardware implementation, unlike full search (FS), and the earlier mentioned algorithms that have a fixed set of search patterns. This is because the advanced algorithms are complicated and rely heavily on thresholds. The possible penalties of these include complicated control circuits, pipeline flushing, sequential processing, and inefficient hardware use.

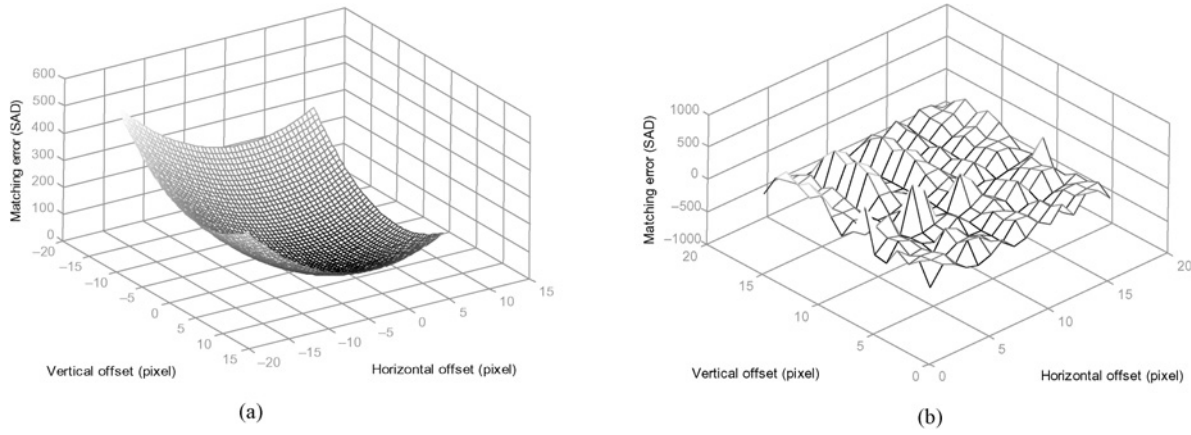


Fig. 1. (a) Uni-modal error surface with a global minimum. (b) Non-uni-modal error surface with multiple local minima.

In terms of hardware ME acceleration, while most hardware architectures have been designed for full search motion estimation (FSME) [14]–[16], some architectures have been proposed for hardware-oriented, fast ME algorithms. Examples of these architectures include Chao *et al.* [17] (based on DS), Miyakoshi *et al.* [18] (based on gradient descent), Lin *et al.* [19] (based on 4SS), Chen *et al.* [20] (based on content-adaptive variable block size 4SS) and Qiu *et al.* [21] (based on adaptive crossed quarter polar patterns). The works in [14]–[16], and [21] are implemented on FPGA-based platforms, while the works in [17]–[20] are implemented as application-specific integrated circuits (ASICs).

In our earlier works [22] and [23], we proposed hardware-oriented modifications to SUMH, as well as supporting hardware architecture. The modifications to SUMH resulted not only in a hardware-oriented fast search algorithm, but also an algorithm that outperforms SUMH. However, because modified SUMH is adapted for hardware from SUMH, it inherits redundant search points. It is our goal to minimize algorithm complexity and related hardware inefficiencies by co-designing a hardware-oriented algorithm along with its supporting hardware architecture. In [24], we have introduced a new hardware-oriented modified diamond search (HMDS) algorithm, for integer-pel, fast motion estimation.

In this paper, we provide details on HMDS. In addition, we propose a co-designed hardware architecture for HMDS. Our algorithm has regular search patterns based on a modified diamond search pattern. This regularity is good for memory organization and data reuse during parallel processing.

Our results show that HMDS outperforms current state-of-the-art algorithms, typically at lower bit rates. Its rate-distortion losses when compared with FSME are insignificant. Our FPGA-based supporting architecture is more hardware efficient than previous FPGA-based architectures [14]–[16], [21]. Furthermore, its performance in terms of power-consumption and minimum frequency is comparable to ASIC-based architectures [17]–[20]. However, whereas our architecture supports H.264/AVC, those in [17]–[20] do not.

The rest of this paper is organized as follows. In Section II we present HMDS. In Section III, we present our simulation results and analysis on HMDS. In Section IV, we present

our proposed architecture for HMDS and our implementation results. Finally our conclusions are presented in Section V.

II. THE PROPOSED HMDS ALGORITHM

The block distortion measure of choice, for integer-pel ME in H.264/AVC is the sum of absolute differences (SAD) given by the mathematical expression in (1)

$$SAD(di, dj) = \sum_{i=0}^{I-1} \sum_{j=0}^{J-1} |x(i, j) - y(i + di, j + dj)| \quad (1)$$

$$(MV_i, MV_j) = (di, dj)_{\min SAD(di, dj)} \quad (2)$$

In (1), $x(i, j)$ and $y(i, j)$ are the pixels of the current and candidate blocks, respectively. (di, dj) is the displacement of the candidate block within the search window. $I \times J$ is the size of the current block. In (2) (MV_i, MV_j) is the motion vector of the best matching candidate block.

Our proposed HMDS algorithm is based on a modified diamond search pattern. After careful study of various basic shape search patterns, we concluded that the diamond shape is extremely efficient at finding true motion vectors. In [10] and [11], more than 31 standard test sequences were used to study the average motion vector probability for different search shape patterns. Table I summarizes their findings. From Table I we see that the square shape pattern has the highest cumulative probability of motion vector occurrence. Full search is based on the square search shape pattern. The next most efficient shape is the diamond, with only slight loss in search efficiency, especially for low resolution video. The diamond shape has high search efficiency because most video contains translation, zooming, pan and tilt motions, which easily fall on the diamond corners when block ME is employed. However, diamond search is still susceptible to finding local minima and missing the globally optimum motion vector. In order to further improve search efficiency, we propose the modified diamond pattern shown in Fig. 2(a).

In Fig. 2(a), the outermost search points are at the search range boundaries. These satellite search points make the diamond search robust enough to capture large motions. Thus

TABLE I
AVERAGE CUMULATIVE PROBABILITY OF MOTION VECTOR
OCCURRENCE FOR VARIOUS SEARCH PATTERNS (%)

Resolution, Search Range	Square	Diamond	Cross
CIF, ± 7	100	97.03	83.48
CCIR601, ± 15	100	92.00	67.14

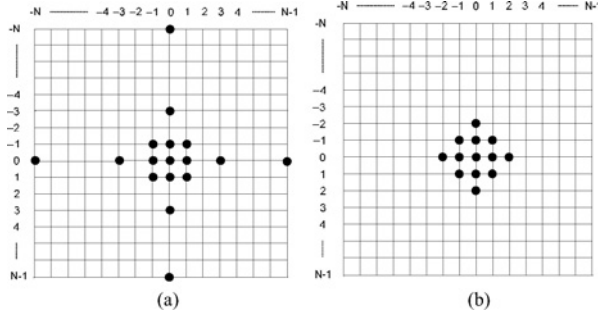


Fig. 2. HMDS search patterns. (a) Modified diamond search pattern. (b) Large diamond search pattern (LDSP).

TABLE II
ALGORITHM: PROPOSED HMDS MOTION ESTIMATION ALGORITHM

Step 1 :	The minimum block distortion measure is found from the search points in the modified diamond search pattern shown in Fig. 2(a).
Step 2 :	With the minimum block distortion measure in Step 1 as the center, a LDSP is formed as in Fig. 2(b). Identify the new minimum BDM from the candidate search points.
Step 3 :	With the minimum block distortion measure in Step 2 as the new search center, a LDSP is formed as in Fig. 2(b). Identify the minimum BDM from the candidate search points.

in general HMDS is more likely to find the globally optimum motion vector. The other search pattern we use in HMDS is the well-known large diamond search pattern (LDSP), shown in Fig. 2(b). For motion vector (MV) prediction, HMDS uses the zero displacement and spatial median predictors defined in H.264/AVC.

HMDS is done in three search steps. In the first step, the modified diamond shape in Fig. 2(a) is employed. Subsequent steps search the LDSP in Fig. 2(b), whose center is the search point with the least block distortion measure from the previous step. HMDS terminates after the third search step. The algorithm is outlined in Table II. Its complete flow chart is shown in Fig. 3, while Fig. 4 shows a sample search flow.

III. PERFORMANCE AND ANALYSIS OF HMDS

Our experiments are done in JM 13.2 [25]. We use the following standard test sequences: *Stefan*, *Carphone* (large motion), *Flower* (large motion with high detail and camera panning), *Foreman*, *Coastguard* (large to moderate motion), *Suzie* (moderate motion), *Silent* (small motion), and *Mother-daughter* (small motion, talking head and shoulders). The sequences are coded at 30 Hz. The picture sequence is IPPP with I-frame refresh rate set at every 15 frames. We consider 1 reference frame (our results are similar for multiple reference

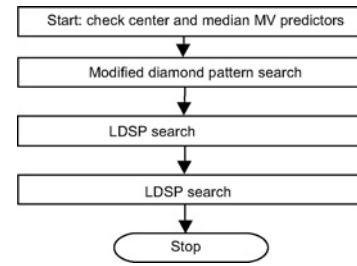


Fig. 3. Flow chart of HMDS.

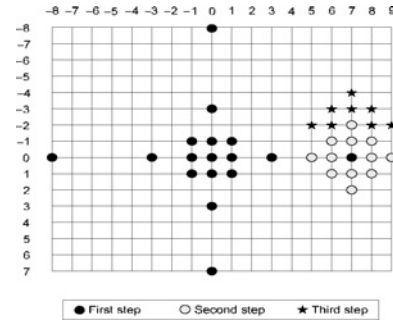


Fig. 4. Example showing a sample search flow of HMDS.

frames). In addition, we consider test conditions similar to the aforementioned, but with lower frame rates that stress any ME algorithm. The sequences used in these tests are *Mobile*, *Hall* (large motion), *Waterfall* (moderate motion with high detail), *Tennis* (large and fast motion), *Salesman* (moderate motion, talking head and shoulders), *Container* (slow and uniform motion), *News*, and *Miss-America* (small motion). Finally, we test with *Football* sequence (large and fast motion, high spatial detail), and *Parkrun* sequence (medium motion, high spatial detail). These last sequences are tested at H.264/AVC main profile, picture sequence IBBPBB, I-frame refresh rate set at every 15 frames, and 2 reference frames.

In all our experiments, rate-distortion optimization is set to high complexity mode (i.e., rate-distortion optimization is turned on), in order to ensure that all of the compared algorithms have a fair chance to yield their highest rate-distortion performance. The rest of our simulation conditions are summarized in Table III.

A. Rate-Distortion Performance of HMDS

From Fig. 5 and Tables IV–IX, we observe that in terms of rate-distortion (RD), performance, HMDS outperforms several well-known fast ME algorithms, including SUMH, typically at low bit rates. Its performance is comparable with that of the modified SUMH and its performance loss is insignificant when compared with full search.

Our detailed results in Tables V and VII show that for HMDS, the average Y-PSNR loss is 0.0018 dB to 0.0283 dB when compared to FSME, -0.0147 dB to 0.001 dB, when compared to SUMH, and -0.0023 dB to 0.0078 dB when compared to modified SUMH.

From Tables VI and VIII, we see that the average percentage bit rate increase of HMDS is very low. HMDS shows a bit rate increase of -0.16% to 1.85% when compared with FSME,

TABLE III
SIMULATION CONDITIONS

Sequences	Quantization Parameter	Search Range	Frame Size	Frame Rate	No. of Frames	Entropy Coding	GOP Format
<i>Foreman</i>	22, 25, 28, 31, 33, 35	32	CIF	30	100	CAVLC	IPPP
<i>Mother-daughter</i>	22, 25, 28, 31, 33, 35	32	CIF	30	150	CAVLC	IPPP
<i>Stefan</i>	22, 25, 28, 31, 33, 35	16	CIF	30	90	CAVLC	IPPP
<i>Flower</i>	22, 25, 28, 31, 33, 35	16	CIF	30	150	CAVLC	IPPP
<i>Coastguard</i>	18, 22, 25, 28, 31, 33	32	QCIF	30	220	CAVLC	IPPP
<i>Carphone</i>	18, 22, 25, 28, 31, 33	32	QCIF	30	220	CAVLC	IPPP
<i>Silent</i>	18, 22, 25, 28, 31, 33	16	QCIF	30	220	CAVLC	IPPP
<i>Suzie</i>	18, 22, 25, 28, 31, 33	16	QCIF	30	220	CAVLC	IPPP
<i>Mobile</i>	18, 22, 26, 30, 34, 38	16	SIF	7.5	220	CAVLC	IPPP
<i>Tennis</i>	18, 22, 26, 30, 34, 38	16	SIF	10	150	CAVLC	IPPP
<i>Waterfall</i>	18, 22, 26, 30, 34, 38	16	CIF	7.5	125	CAVLC	IPPP
<i>Container</i>	18, 22, 26, 30, 34, 38	16	CIF	10	220	CAVLC	IPPP
<i>Hall</i>	18, 22, 26, 30, 34, 38	32	CIF	15	220	CAVLC	IPPP
<i>Salesman</i>	18, 22, 26, 30, 34, 38	16	QCIF	7.5	220	CAVLC	IPPP
<i>News</i>	18, 22, 26, 30, 34, 38	16	QCIF	10	220	CAVLC	IPPP
<i>Miss-America</i>	18, 22, 26, 30, 34, 38	32	QCIF	15	150	CAVLC	IPPP
<i>Football</i>	28	16	CIF	30	90	CABAC	IBBPBBP
<i>Parkrun</i>	28	128	720p	60	30	CABAC	IBBPBBP

TABLE IV
COMPARISON OF AVERAGE Y-PSNR LOSSES COMPARED WITH FULL SEARCH (dB)

Sequence	TSS [2]	FSS [4]	UCB DS [5]	DS [6]	HEX [7]	EPZS [9]	UMH [12]	SUMH [13]	ModSUMH [23]	ACQPPS [21]	HMDS [This]
<i>Foreman</i>	0.58	0.60	0.47	0.39	0.61	0.03	0.06	0.036	0.029	0.05	0.027
<i>Carphone</i>	0.13	0.19	0.27	0.20	0.39	0.01	0.06	0.043	0.021	0.06	0.028
<i>Tennis</i>	0.15	0.15	0.08	0.16	0.16	0.00	0.03	0.013	0.012	0.03	0.010
<i>News</i>	0.16	0.15	0.09	0.18	0.19	0.01	0.09	0.020	0.018	0.01	0.018
<i>Miss-Am</i>	0.42	0.13	0.06	0.41	0.06	0.09	0.05	0.026	0.005	0.04	0.022
<i>Suzie</i>	0.06	0.11	0.18	0.19	0.20	0.04	0.07	0.019	0.013	0.05	0.011

TABLE V
AVERAGE Y-PSNR LOSS FOR HMDS

Compared With Sequences	Full Search	SUMH	Modified SUMH
<i>Foreman</i>	0.0272 dB	−0.0083 dB	−0.0018 dB
<i>Stefan</i>	0.0103 dB	−0.0080 dB	0.0045 dB
<i>Mother-daughter</i>	0.0198 dB	−0.0008 dB	0.0012 dB
<i>Flower</i>	0.0018 dB	−0.0025 dB	−0.0023 dB
<i>Coastguard</i>	0.0065 dB	−0.0005 dB	−0.0013 dB
<i>Silent</i>	0.0082 dB	0.0002 dB	−0.0017 dB
<i>Carphone</i>	0.0283 dB	−0.0147 dB	0.0078 dB
<i>Suzie</i>	0.0110 dB	−0.0075 dB	−0.0023 dB

TABLE VI
AVERAGE PERCENTAGE BIT RATE INCREASE FOR HMDS

Compared With Sequences	Full Search	SUMH	Modified SUMH
<i>Foreman</i>	1.26	−0.07	−0.03
<i>Stefan</i>	1.13	0.38	0.72
<i>Mother-daughter</i>	0.59	0.38	0.43
<i>Flower</i>	0.25	−0.11	0.06
<i>Coastguard</i>	0.03	0.01	0.04
<i>Silent</i>	0.86	−0.04	0.29
<i>Carphone</i>	1.24	0.90	0.96
<i>Suzie</i>	0.63	−0.08	0.63

TABLE VII
AVERAGE Y-PSNR LOSS FOR HMDS (LOW FRAME RATES)

Compared With Sequences	Full Search	SUMH	Modified SUMH
<i>Mobile</i>	0.0113 dB	0.0007 dB	0.0010 dB
<i>Tennis</i>	0.0102 dB	−0.0023 dB	−0.0015 dB
<i>Waterfall</i>	0.0077 dB	0.0008 dB	0.0020 dB
<i>Container</i>	0.0153 dB	−0.0038 dB	0.0023 dB
<i>Hall</i>	0.0205 dB	−0.0087 dB	0.0007 dB
<i>Salesman</i>	0.0127 dB	0.0010 dB	0.0010 dB
<i>News</i>	0.0177 dB	−0.0027 dB	0.0002 dB
<i>Miss-America</i>	0.0217 dB	−0.0047 dB	0.0048 dB

−0.11% to 0.90% when compared with SUMH, and −0.16% to 0.96% when compared with modified SUMH.

From Table IX, we find that the bit rate increases slightly with the introduction of B-frames and at larger search ranges.

From Fig. 5 and Tables V and VII, we also observe that when HMDS is compared with full search, larger Y-PSNR losses occur in those motion sequences where the bit rate is somewhat mismatched with the degree of motion. Examples of these mismatches include:

- 1) occurrences of large motion within relatively low bit rates (or high bit rate constraints), such as in *Carphone*, *Foreman*, and *Salesman* sequences;

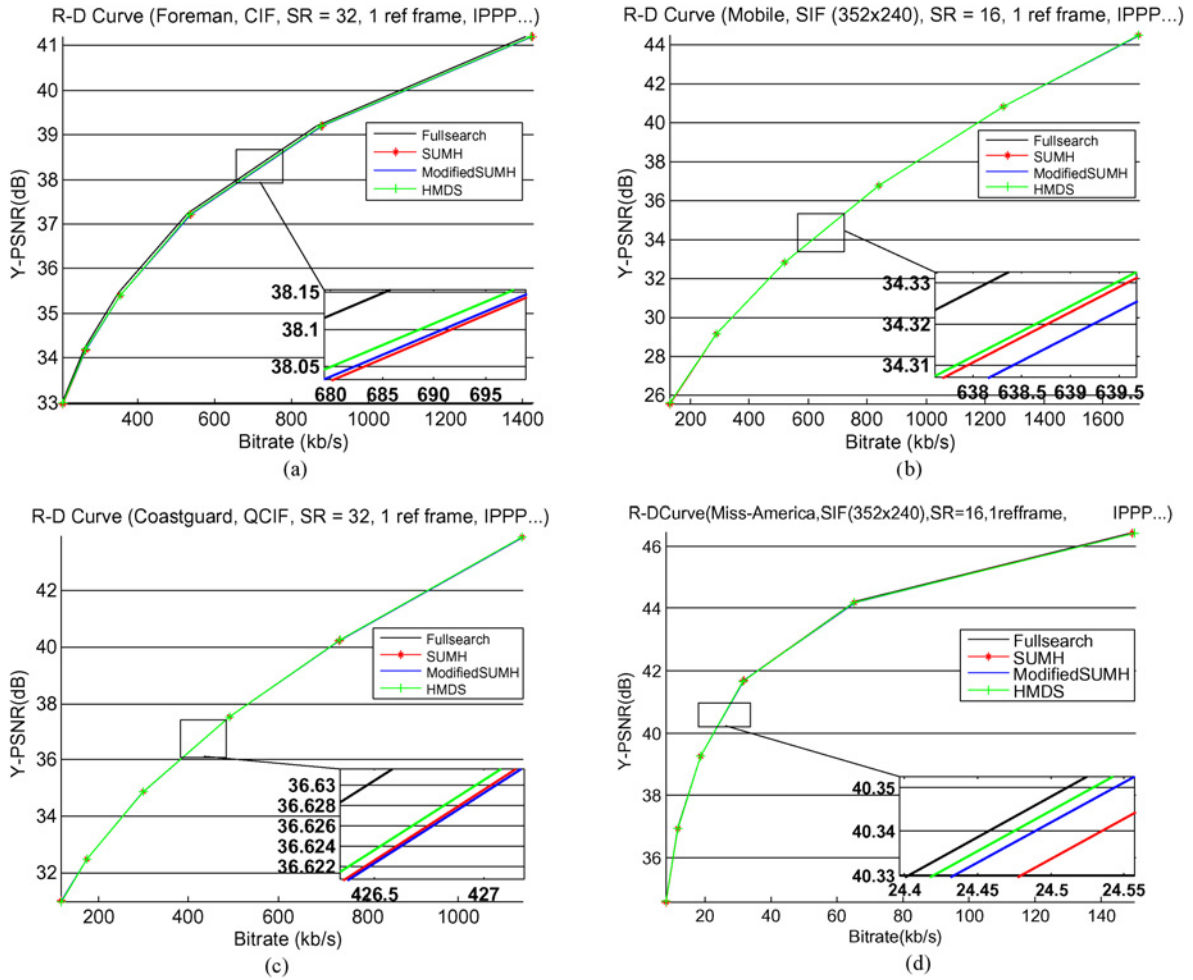


Fig. 5. Comparison of rate-distortion efficiencies for HMDS using different test sequences. (a) *Foreman*. (b) *Mobile*. (c) *Coastguard*. (d) *Miss America*.

TABLE VIII
AVERAGE PERCENTAGE BIT RATE INCREASE FOR HMDS (LOW FRAME RATES)

Compared With Sequences	Full Search	SUMH	Modified SUMH
<i>Mobile</i>	0.02	-0.04	-0.09
<i>Tennis</i>	1.85	0.73	0.68
<i>Waterfall</i>	-0.06	-0.05	0.00
<i>Container</i>	-0.01	-0.01	0.02
<i>Hall</i>	0.12	0.03	0.14
<i>Salesman</i>	0.26	-0.11	-0.04
<i>News</i>	0.13	-0.03	-0.16
<i>Miss-America</i>	-0.16	0.12	-0.02

- 2) occurrences of moderate to small motion within relatively large bit rates, such as *Mother-daughter*, *Suzie*, *Miss-America*, *Hall*, *Container*, and *News*.

In the remaining sequences *Stefan*, *Flower*, *Coastguard*, *Silent*, *Mobile*, *Tennis*, and *Waterfall*, the Y-PSNR losses are less. This is because there is relatively less motion between objects in those sequences.

We see from the graphs in Fig. 5 that with few exceptions, HMDS consistently outperformed SUMH at lower bit rates

(≤ 90 kb/s). We earlier surmised in [23] that SUMH is robust against the local minima problem and only needed improvement in its local search. HMDS is also robust against the local minima problem but in addition we improved local search. This is the reason why HMDS outperforms SUMH at low bit rates.

B. Speedup Performance of HMDS

The speedup performance of HMDS is shown in Tables X, XI, and XII, expressed as percentage time savings when compared to full search.

From Table X, we see that HMDS yields more speedup than many fast ME algorithms. Its speedup is only surpassed occasionally, by DS. From detailed results in Tables XI and XII, we find that SUMH saves 85.5% to 98.8% in ME computation time compared to full search, the modified SUMH saves 32.2% to 91.7% in ME computation time compared to full search, and HMDS saves 71.0% to 97.6%. Comparing the averages of these ranges, we find that HMDS loses an average of 7.85% in speedup, when compared with SUMH. On the other hand, HMDS gains an average of 22.35% in speedup when compared with the modified SUMH. It is also noteworthy that there are large variations in speedup values of the modified SUMH, while HMDS yields a fairly narrow range of speedup values.

TABLE IX
PERFORMANCE COMPARED WITH FULL SEARCH

Sequence	Resolution	SearchRange	Algorithm	Y-PSNR	% Bitrate Increase	% Speedup
Football	CIF 352×288	16	FS	37.424	0	1
Football	CIF 352×288	16	SUMH	37.432	2.43	87.44
Football	CIF 352×288	16	HMDS	37.422	5.57	86.42
Parkrun	HD 720p@60	128	FS	34.548	0	1
Parkrun	HD 720p@60	128	SUMH	34.549	0.21	99.38
Parkrun	HD 720p@60	128	HMDS	34.555	2.20	99.70

CABAC, GOP = IBBPBBP...

TABLE X
COMPARISON OF AVERAGE NUMBER OF SEARCH POINTS PER MB (SEARCH RANGE ± 16)

Sequence	FS	TSS [2]	FSS [4]	UCBDS [5]	DS [6]	HEX [7]	EPZS [8]	UMH [11]	SUMH [12]	Mod SUMH [22]	ACQPPS [20]	HMDS [This]
Foreman	1089	129	126	125	77	117	122	124	66	113	67	41
Carphone	1089	108	107	92	47	95	115	122	66	113	54	41
Tennis	1089	120	121	106	56	109	128	125	66	113	54	41
News	1089	88	90	74	34	81	81	80	66	113	41	41
Miss-Am	1089	75	77	64	31	68	63	56	66	113	32	41
Suzie	1089	108	105	88	44	93	97	89	66	113	48	41

In general therefore, at low bit rates, the performance losses obtained when HMDS is compared with full search are insignificant, while on the other hand they are comparable with the modified SUMH, and there is an improvement when compared with SUMH. HMDS is thus suitable for H.264/AVC baseline profile at levels 1–2 (i.e., low bit rate QCIF–SIF video sequences and search ranges of up to ± 32).

As discussed in Section II, HMDS is robust enough to find the globally optimum motion vector because it can simultaneously track large motion using its satellite search points, and local motion through its last two steps of intensive local search.

C. Complexity Analysis of the Motion Estimation Algorithms

We consider a search range s . The number of search points to be examined by FSME algorithm is directly proportional to the square of the search range. There are $(2s + 1)^2$ search points. Thus, the algorithm complexity of full search is $O(s^2)$.

We analyze the algorithm complexity of SUMH [13] algorithm by considering the algorithm complexity of each of its search steps as follows.

- 1) Cross search: There are s search points both horizontally and vertically yielding a total of $2s$ search points. Thus the algorithm complexity of this search step is $O(2s)$.
- 2) Hexagon and extended hexagon search: There are 6 search points each in both of these search steps, yielding a total of 12 search points. Thus, the algorithm complexity of this search step is constant $O(1)$.
- 3) Multibig hexagon search: There are $\frac{1}{4}s$ hexagons with 16 search points per hexagon. This yields a total of $4s$ search points. Thus the algorithm complexity of this search step is $O(4s)$.
- 4) Diamond search: There are four search points in this search step. Thus, the algorithm complexity of this search step is constant $O(1)$.

In total there are $1 + 2s + 12 + 4 + 4s$ search points in the modified SUMH [23], and its algorithm complexity is $O(6s)$.

In order to obtain the algorithm complexity of SUMH, we consider its worst case complexity, even though the algorithm may terminate much earlier. The worst case complexity of SUMH is similar to that of the modified SUMH, except that it adds 14 more search points. This number is obtained by adding four search points each for two small local searches and one convergence search, and two search points for the worst case up layer predictor search. Thus for the worst case SUMH, there are in total $14 + 1 + 2s + 12 + 4 + 4s$ search points in the modified SUMH, and its algorithm complexity is $O(6s)$. Note that in the best case, SUMH has only five search points: one for the initial search candidate and four for the convergence search.

The algorithmic complexity of HMDS is constant $O(1)$ and also it is also lower than that of full search, SUMH and the modified SUMH. There are 41 search points in HMDS. Note, however, that in a worst case search flow (e.g., Fig. 4), only 36 of the 41 search points are unique. Five search points in the last search step are repeated.

Another way to define the complexity of each algorithm is in terms of the number of required operations. We can then express the complexity as million operations per second (MOPS). To compare the algorithms in terms of MOPS we assume the following.

- 1) The macroblock size is 16×16 .
- 2) The SAD cost function requires $2 \times 16 \times 16$ data loads, $16 \times 16 = 256$ subtraction operations, 256 absolute operations, 256 accumulate operations, 41 compare operations, and 1 data store operation. This yields a total of 1322 operations for one SAD computation.
- 3) CIF resolution is 352×288 pixels = 396 macroblocks.
- 4) The frame rate is 30 frames/s.
- 5) The total number of operations required to encode CIF video in real time is $1322 \times 396 \times 30 \times z_a$, where z_a is the number of search points for each algorithm.

TABLE XI

COMPARISON OF PERCENTAGE TIME SAVINGS WITH FULL SEARCH

Sequence	Quantization Parameter	SUMH	Modified SUMH	HMDS
<i>Foreman</i>	22	97.94	87.75	96.29
	25	97.59	85.43	95.54
	28	96.94	82.34	94.68
	31	96.13	79.04	93.63
	33	95.38	76.36	92.70
<i>Stefan</i>	35	94.76	73.31	91.95
	22	93.48	78.83	88.51
	25	92.40	76.29	87.92
	28	91.80	74.61	87.07
	31	90.63	71.46	85.71
<i>Mother-Daughter</i>	33	90.05	69.05	84.31
	35	88.83	67.35	83.67
	22	93.98	59.92	88.92
	25	94.82	63.34	89.51
	28	95.36	66.85	90.61
<i>Flower</i>	31	96.50	71.22	91.93
	33	97.17	76.21	93.23
	35	97.72	80.35	94.57
	22	89.72	67.45	84.21
	25	90.67	69.62	85.12
<i>Coastguard</i>	28	91.16	71.37	85.81
	31	92.27	73.56	86.63
	33	92.71	75.14	87.89
	35	93.34	76.27	88.45
	18	98.84	91.71	97.63
<i>Silent</i>	22	98.57	90.30	97.17
	25	98.27	88.91	96.86
	28	97.70	87.47	96.46
	31	97.22	85.29	95.93
	33	96.67	83.70	95.48
<i>Carphone</i>	18	95.42	71.77	86.13
	22	94.02	68.62	84.91
	25	92.40	66.61	84.14
	28	91.60	64.56	83.24
	31	89.23	62.47	82.60
<i>Suzie</i>	33	88.32	62.20	83.02
	18	95.94	75.87	93.18
	22	96.60	78.36	93.82
	25	97.30	81.41	94.54
	28	97.87	83.41	95.35
<i>Miss-America</i>	31	98.14	85.87	96.07
	33	98.43	88.66	96.72
	18	88.00	53.08	77.04
	22	88.74	57.45	79.76
	25	90.70	64.92	82.27
<i>News</i>	28	92.72	68.63	84.79
	31	94.05	73.00	86.39
	33	94.80	75.71	88.42

TABLE XII

COMPARISON OF PERCENTAGE TIME SAVINGS WITH FULL SEARCH (LOW FRAME RATES)

Sequence	Quantization Parameter	SUMH	Modified SUMH	HMDS
<i>Mobile</i>	18	89.46	72.04	85.52
	22	91.35	75.15	87.28
	26	92.53	78.21	89.02
	30	93.40	80.07	89.98
	34	93.99	81.64	90.71
<i>Tennis</i>	38	94.49	82.68	91.00
	18	86.17	59.51	79.91
	22	88.82	66.53	83.38
	26	90.98	72.04	85.51
	30	92.78	75.59	87.95
<i>Waterfall</i>	34	94.14	78.64	89.43
	38	95.02	81.44	91.00
	18	86.89	61.54	80.26
	22	88.63	67.49	83.19
	26	91.06	72.48	85.97
<i>Container</i>	30	93.12	76.92	88.02
	34	94.53	79.19	89.68
	38	95.51	81.75	90.81
	18	97.29	42.90	72.23
	22	97.44	52.88	77.18
<i>Hall</i>	26	97.48	61.04	80.51
	30	92.80	67.56	84.15
	34	94.23	72.12	86.72
	38	95.37	76.16	88.80
	18	92.51	50.75	86.14
<i>Salesman</i>	22	94.60	61.04	88.91
	26	96.39	70.80	91.75
	30	97.41	77.97	93.74
	34	98.14	84.72	95.52
	38	98.63	89.71	96.96
<i>News</i>	18	86.14	51.94	77.40
	22	87.54	55.62	79.07
	26	91.04	61.73	81.48
	30	93.13	64.80	83.96
	34	94.23	68.29	85.29
<i>Miss-America</i>	38	95.92	72.54	88.41
	18	85.48	41.26	71.02
	22	87.85	48.01	75.21
	26	90.74	53.70	78.15
	30	92.36	58.24	80.23
<i>Suzie</i>	34	94.12	63.67	83.27
	38	94.75	68.66	85.85
	18	91.97	32.22	82.70
	22	94.20	45.65	85.73
	26	95.97	57.82	88.69
<i>Carphone</i>	30	96.86	68.17	91.53
	34	97.97	76.11	93.66
	38	98.40	81.70	95.10

Thus there are $15.7 z_a$ MOPS per algorithm, where one operation (OP) is the amount of computation it takes to obtain one SAD value.

In Table XIII, we compare the computational complexities of the considered algorithms in terms of MOPS. As expected, FSME requires the largest number of MOPS. The number of MOPS required for the modified SUMH is about 10% less than that required for the worst case SUMH and about 40% more than that required for the median case SUMH.

We also observe from Table XIII that the number of MOPS required for HMDS is 96% less than that required for FSME. It is also about 68% less than that required for the worst case SUMH, 38% less than that required for the median case SUMH and about 64% less than what is required for the modified SUMH. It is interesting to note from Tables XI and XII that in practice, HMDS incurred an average speedup penalty of 7.85%, when compared to SUMH. This means that SUMH

TABLE XIII
COMPLEXITY OF ALGORITHMS IN MILLION OPERATIONS PER SECOND (MOPS)

Algorithm	Number of Search Points for Search Range $s = \pm 16$	Number of MOPS for CIF Video at 30 Hz
FSME	1089	17 103
Best case SUMH	5	78
Worst case SUMH	127	1995
Median case SUMH	66	1037
Modified SUMH	113	1775
HMDS	41	643

TABLE XIV
SEARCH PASSES FOR HMDS

Pass	Description
1–4	HMDS search step 1: first search pattern of HMDS
5–7	HMDS search step 2: second search pattern of HMDS
8–10	HMDS search step 3: second search pattern of HMDS

TABLE XV
DATA SCHEDULE FOR PROCESSING UNIT (PU)

Clock	PU1	...	PU4	Comments
1-16	(-1, 0)-(-14, 0) ⋮ (-1, -15)-(-14, -15)	...	(1, 0)-(16, 0) ⋮ (1, -15)-(16, -15)	Search pass 1: scan of inner Greek cross of search step 1
17-32	(-1, -1)-(-14, -1) ⋮ (-1, -14)-(-14, -14)	...	(1, 1)-(16, 1) ⋮ (1, -16)-(16, -16)	Search pass 2: scan of St. Andrew cross of search step 1
33-48	(-3, 0)-(-12, 0) ⋮ (-3, -15)-(-12, -15)	...	(3, 0)-(14, 0) ⋮ (13, -15)-(14, -15)	Search pass 3: scan of middle Greek cross of search step 1
49-64	(-16, 0)-(-1, 0) ⋮ (-16, -15)-(-1, -15)	...	(15, 0)-(30, 0) ⋮ (15, -15)-(30, -15)	Search pass 4: scan of outer Greek cross of search step 1
⋮	⋮	⋮	⋮	⋮

often terminates earlier than its median number of search points.

IV. THE PROPOSED ARCHITECTURE FOR HMDS

Our top-level architecture for fast integer VBSME is shown in Fig. 6. The architecture is composed of two ping-pong banks of search window (SW) memory, a switch box, current MB memory, an address generation unit (AGU), a control unit, a block of processing units (PU), a SAD combination tree, a comparison unit and a register for storing the 41 minimum SADs and associated motion vectors of H.264/AVC.

While the current and reference frames are stored off-chip in external memory, the current MB (CMB) data and the SW data are stored in on-chip, dual-port block RAMS (BRAMS).

The SW memory has $8 \times 16 \times 16$ BRAMs arranged in 2 banks of 4 BRAMS each. Each memory bank stores 4 candidate MBs. A switch box (which is a multiplexer) switches between the ping-pong buffers. The AGU generates addresses for blocks being processed.

There are 4 PUs, each containing 16 processing elements (PE), in a 1-D array. A PU shown in Fig. 7 calculates 164×4 SADs for one candidate MB while a PE shown in Fig. 9 calculates the absolute difference between two pixels, one each from the candidate MB and the current MB. From Fig. 7, groups of 4 PEs in the PU calculate 1 column of 4×4 SADs. These are stored via demultiplexing, in registers D1–D4 which hold the inputs to the SAD combination tree, one of which is shown in Fig. 8. For four PUs there are four SAD combination trees. Each SAD combination tree further combines the 164×4 output SADs from one PU to yield a total of 41 SADs per candidate MB. Fig. 8 shows that the 164×4 SADs are combined such that registers D6 contain 4×8 SADs, D7 contain 8×8 SADs, D8 contain 8×16 SADs, D9 contain 16×8 SADs, D10 contain 8×4 SADs, and finally D11 contains the 16×16 SAD. These SADs are compared appropriately in the comparison unit (CU). CU consists of 41 4-input comparing elements (CE). A CE is shown in Fig. 10.

A. Address Generation Unit

For each of 4 MBs being processed simultaneously, the AGU generates the addresses of the top row and the leftmost column of 4×4 sub-blocks. The address of each sub-block is the address of its top left pixel. From the addresses of the top row and leftmost column of 4×4 sub-blocks, we obtain the addresses of all other block partitions in the MB.

The interface of the AGU is parameterized by the address of the current MB and the search pass. The search pass depends on the search step and the search range. We show in Table XVI that there are ten search passes for HMDS, regardless of the value of search s .

B. Memory

Figs. 11 and 12 show CMB and SW memory organization for 4 PUs. Both CMB and SW memory are synthesized into BRAMs. Regardless of the value of search range s , there are ten search passes for HMDS search flowchart shown in Fig. 3. These search passes are shown in Table XIV. In each search pass, 4 MBs are processed in parallel, hence the SW memory organization shown in Fig. 12, for one memory bank. Each memory bank is 64 bytes wide and the size is 1024 bytes. This means that given 2 memory banks, the total memory size is 2048 bytes. For a search range $s = \pm 16$, if FSME were used along with level A and B data reuse, the SW memory size would be 48×48 pixels, i.e., 2304 bytes [14]. Thus by using HMDS, we achieve an 11% on-chip memory savings even without a data reuse scheme.

In each clock cycle, we load 64 bits of data, constrained by our FPSoC bus width. Therefore, it takes 128 cycles to load data for one search pass and 1280 (128×10) cycles to load data for one CMB. Under similar conditions for FSME it would take 288 (6×48) clock cycles to load data for one CMB.

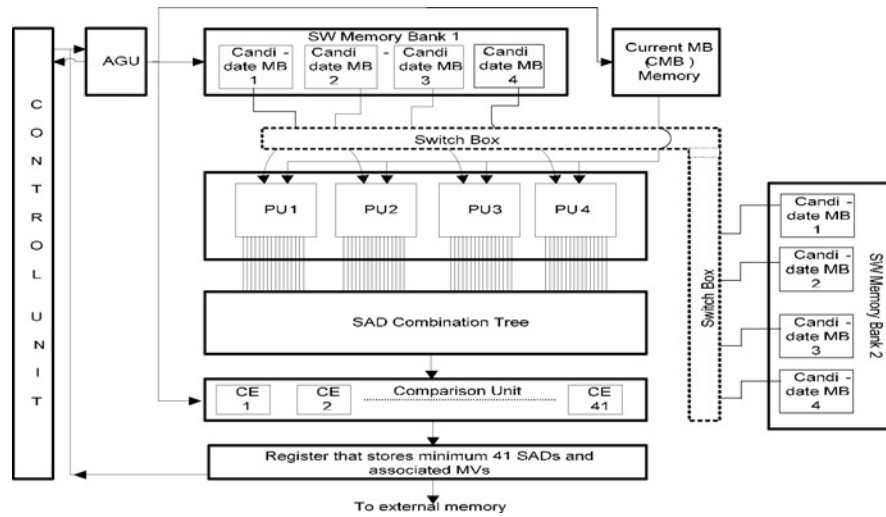


Fig. 6. Proposed architecture for HMDS.

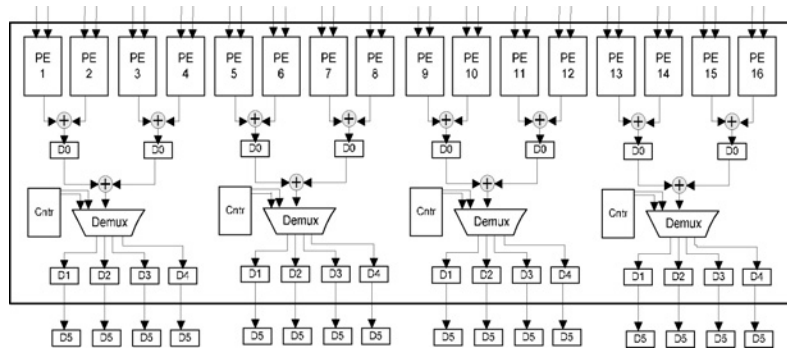


Fig. 7. Architecture of a processing unit (PU).

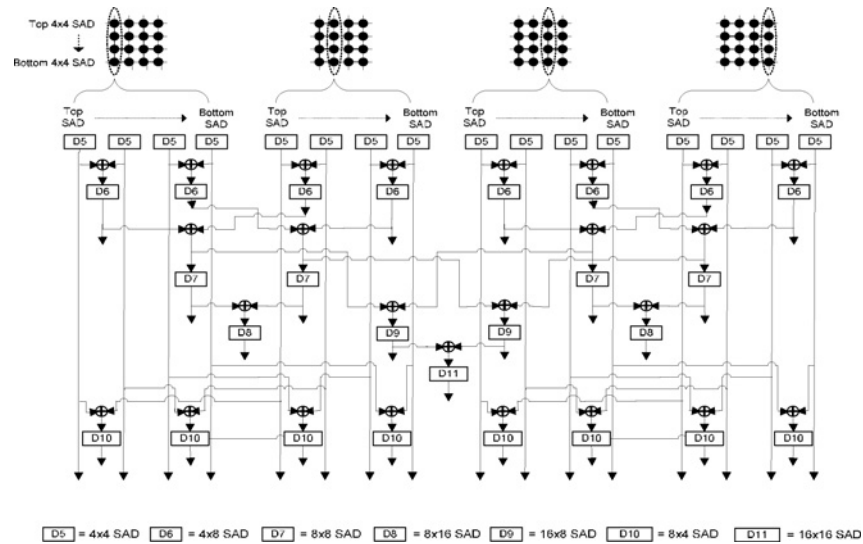


Fig. 8. SAD combination tree.

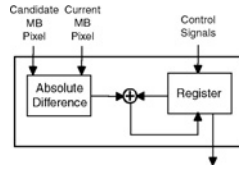


Fig. 9. Processing element (PE).

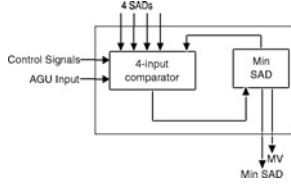


Fig. 10. Comparing element (CE).

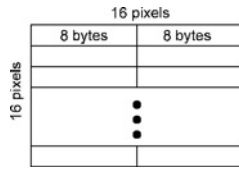


Fig. 11. Data arrangement in current macroblock (CMB) memory.

Thus, the ratio of the required memory bandwidth for HMDS to the required memory bandwidth for FSME is 4.4. While this ratio is undesirably greater than 1, it is well mitigated by the fact that there are only 41 search locations for one CMB in HMDS, compared to 1089 search locations for one CMB in FSME. In other words, the amount of computation for one CMB in HMDS is approximately 0.04 that for FSME. Furthermore, with FSME and level A and B data reuse, there would be a need for lot of broadcast (multi-output) nets or a lot of shift registers. Broadcast nets increase power consumption since the power consumption of a net is directly proportional to the output load that it drives. The use of shift registers suffers the penalty of an increase in power consumption as well as the number of clock cycles it takes to process one current MB. Thus, there is an overall speed and power savings in using HMDS instead of FSME.

It is noteworthy to observe that in our architecture proposed for the modified SUMH in [23], we utilized 8 PUs and a single bank of memory. This was the optimum architecture for the modified SUMH, but would be a sub-optimum architecture for HMDS. The reason is because both algorithms have a different number of search points per search step. If we utilized the architecture of [23] for HMDS, then there would be 6 search passes instead of the 10 shown in Table XIV. There would be 4 redundant PUs in 2 of the 6 search passes. It would take 256 cycles to load data for one search pass, instead of 128 cycles. Finally, it would take 1536 (256×6) cycles to load data for one CMB, instead of 1280 cycles. Therefore overall, the architecture would be less efficient for HMDS, than the currently proposed architecture in Fig. 6. However it could achieve a lower minimum frequency. We find that 4 PUs yields the best tradeoff between minimum frequency and hardware utilization.

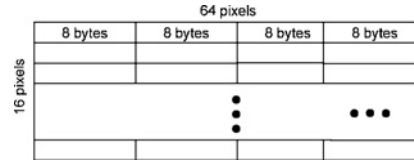


Fig. 12. Data arrangement in search window (SW) memory.

C. Processing Unit

Table XV shows the pixel data schedule for four search passes of the 4 PUs. In Table XV we are considering as an illustration, the first search step and a search range $s = \pm 16$, hence the given pixel coordinates. Table XV shows that it takes 16 cycles to output the 164×4 SADs from each PU.

D. SAD Combination Tree

The data schedule for the SAD combination is shown in Table XVI. There are four SAD combination (SC) trees, each processing 164×4 SADs that are output from each PU. It takes five cycles to combine the 164×4 SADs and output 41 SADs for the seven inter-prediction block sizes in H.264/AVC: 1 16×16 SAD, 2 16×8 SADs, 2 8×16 SADs, 4 8×8 SADs, 8 8×4 SADs, 8 4×8 SADs, and 16 4×4 SADs.

E. Comparison Unit

The data schedule for the CU is shown in Table XVII. The CU consists of 41 CE, each element processing 4 SADs of the same inter-prediction block size from the 4 PUs. Each CE compares SADs in twos. It takes 4 cycles to output the 41 minimum SADs.

F. Field Programmable System-on-Chip

The top-level prototype design of our architecture is shown in Fig. 13. It is based on the prototype in [14]. In [14], Canals *et al.* proposed an FPSoC for full search block matching algorithm. Their implementation is done on a Virtex-4 FPGA.

Our prototype is done on the XUPV2P development board. The board contains a Virtex-II Pro XC2VP30 FPGA with 30816 logic cells, 136 18-bit multipliers, 2448 kb of block RAM, and two PowerPC processors. There is a serial RS-232 port for communication with a host personal computer. The board also features DDR SDRAM DIMM with 512 MB SDRAM installed.

The embedded development tool used to design our prototype is the Xilinx platform studio, in the Xilinx embedded development kit (EDK). The EDK makes it relatively simple to integrate user intellectual property (IP) cores as peripherals in an FPSoC. Hardware/software co-simulation can then be done to test the user IP.

In our prototype design, as shown in Fig. 13, we employ a PowerPC hardcore embedded processor, as our controller. The boot program memory is a 64 kb BRAM. It contains a bootloop program necessary to keep the processor in a known state after we load the hardware and before we load the software. The processor sends stimuli to the ME IP core and reads results back for comparison.

TABLE XVI
DATA SCHEDULE FOR SAD COMBINATION (SC) UNIT

Clock	SC1	SC2	...	SC4
17	16 4 × 4 SAD	16 4 × 4 SAD		16 4 × 4 SAD
18	8 4 × 8 SAD 8 8 × 4 SAD	8 4 × 8 SAD 8 8 × 4 SAD		8 4 × 8 SAD 8 8 × 4 SAD
19	4 8 × 8 SAD	4 8 × 8 SAD		4 8 × 8 SAD
20	2 8 × 16 SAD 2 16 × 8 SAD	2 8 × 16 SAD 2 16 × 8 SAD		2 8 × 16 SAD 2 16 × 8 SAD
21	1 16 × 16 SAD	1 16 × 16 SAD		1 16 × 16 SAD

TABLE XVII
DATA SCHEDULE FOR COMPARISON UNIT (CU)

Clock	CE1 – CE16	CE17 – CE32	CE33 – CE36	CE37 – CE40	CE41
22	8 4 × 4 SAD	8 4 × 8 SAD 8 8 × 4 SAD	8 8 × 8 SAD	8 8 × 16 SAD 8 16 × 8 SAD	8 16 × 16 SAD
23	4 4 × 4 SAD	4 4 × 8 SAD 4 8 × 4 SAD	4 8 × 8 SAD	4 8 × 16 SAD 16 × 8 SAD	4 16 × 16 SAD
24	2 4 × 4 SAD	2 4 × 8 SAD 2 8 × 4 SAD	2 8 × 8 SAD	2 8 × 16 SAD 2 16 × 8 SAD	2 16 × 16 SAD
25	1 4 × 4 SAD	1 4 × 8 SAD 1 8 × 4 SAD	1 8 × 8 SAD	1 8 × 16 SAD 1 16 × 8 SAD	1 16 × 16 SAD

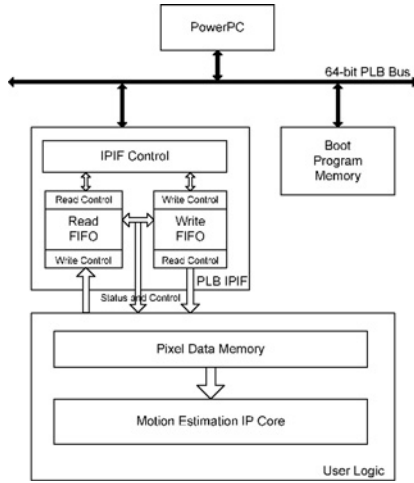


Fig. 13. FPSoC prototype design of our architecture.

The processor is connected to the other design modules, via a 64-bit processor local bus (PLB). The PLB DDR controller provides a control interface for a connected DDR SDRAM, which stores frame data. The PLB also connects to the user IP core through an IP interface, which exposes several programmable interconnects. We employ a PLB BRAM interface controller which interfaces between the PLB and our block RAM FIFO used to store current and reference macroblocks for each search pass. We then implement PLB interfaces as part of our user logic, with burst transaction mode enabled. In each burst transaction, 64 bits of data is read from, or written to the PLB. Status and control signals of the FIFO are available to the user logic block.

The PowerPC is programmed with a table of addresses for CMB and candidate macroblocks for each search pass, given a base address. During operation, the PowerPC processor sends address range of a macroblock to the PLB DDR and

BRAM controllers, and sets FIFO status and control bits. The macroblock data is then transferred from SDRAM to FIFO in the IP. After each transfer, address ranges and status and control signals are updated, until one IP memory bank is filled. ME computation is done while the next memory bank is filled via switching in the switch box. After ME computation is done, the results are written back on the FIFO and the PowerPC reads the results for comparison. Intermediate results are sent to a terminal on the host personal computer, for system verification.

G. Summary of Dataflow

The dataflow represented by the data schedules described variously in Tables XV–XVII may be summarized by the algorithmic state machine (ASM) chart shown in Fig. 14. The ASM chart also represents the mapping of HMDS algorithm in Fig. 3, to our proposed architecture in Fig. 6.

From Fig. 14, we see that implementation of HMDS on our proposed architecture IP core starts in state S1 when the MV, predictors are checked. This is done by the PowerPC processor which is part of our FPSoC in Fig. 13.

The MV predictors are stored in external memory and accessed from there by the PowerPC processor. The output from state S1 is the MV predictors. In the next state S2, the minimum MV cost is obtained and mode decision is done to obtain the right blocktype. This is also done by the PowerPC processor and the outputs of this state are the minimum MV, its SAD cost, its blocktype and its address. The minimum MV cost is obtained by minimizing the cost in (15)

$$J_{\text{motion}}(\vec{m}, REF|\lambda_{\text{motion}}) = SAD(dx, dy, REF, \vec{m}) + \lambda_{\text{motion}} \cdot (R(\vec{m} - \vec{p}) + R(REF)) \quad (3)$$

where $\vec{m} = (m_x, m_y)^T$ is the current MV being considered, REF denotes the reference picture, λ_{motion} is the Lagrangian multiplier, $\vec{p} = (p_x, p_y)$ is the MV used for the prediction,

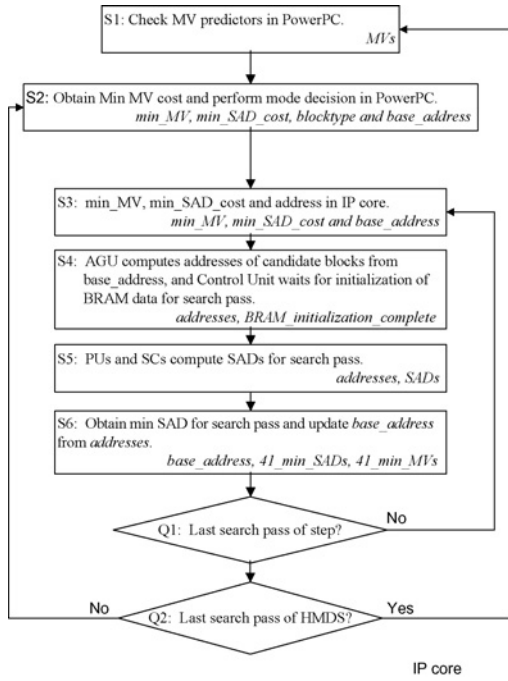


Fig. 14. Algorithmic state machine chart for the HMDS algorithm.

$R(\vec{m} - \vec{p})$ represents the number of bits used for MV coding and $R(REF)$ is the bits for coding REF , and $SAD(d_x, d_y, REF, \vec{m})$ is the SAD cost obtained as in (16)

$$SAD(dx, dy) = \sum_{x=0}^{X-1} \sum_{y=0}^{Y-1} |a(x, y) - b(x + dx, y + dy)| \quad (4)$$

where $a(x, y)$ and $b(x, y)$ are the pixels of the current, and candidate blocks, respectively. (dx, dy) is the displacement of the candidate block within the search window.

In the state S3, some of the outputs from state S2 are passed into our proposed architecture IP core. In state S4, the AGU computes the addresses of candidate blocks, using the address of the MV predictor as the base address, and the control unit waits for the initialization of search window data in the BRAMs. The output of state S4 is the addresses of the candidate blocks and a flag indicating that initialization is complete.

In state S5, the processing units and SAD combination trees compute the SADs of the candidate blocks. The output of S5 is the computed SADs and unchanged AGU addresses. In state S6, the CU compares these SADs with previously computed SADs and obtains the 41 minimum SADs. The outputs of S6 are the 41 minimum SADs and their corresponding addresses.

In the decision block Q1, we check if the current search pass is the last search pass of a particular search step, e.g., the first search step. If no, we continue with other passes of that search step. If yes, we go to decision block Q2. In Q2 we check if it is the last search pass of the HMDS algorithm. If no, we move onto the next search step, e.g., second search step. If yes, we check for the MV predictors of the next current macroblock, according to the HF3V2 2-stitched zigzag scan proposed in [26].

TABLE XVIII
SYNTHESIS RESULTS

Supported algorithm	Modified SUMH	HMDS
Process (μm)	0.13 (FPGA)	0.13 (FPGA)
Number of slices	11.4 K	7.8 K
Number of slice flip flops	16.4 K	11.3 K
Number of 4-input LUTs	18.7 K	10.8 K
Total equivalent gate count	388 K	268 K
Max frequency (MHz)	145.2	246.5
Algorithm	Modified SUMH	HMDS
Video specifications	CIF 30 frames/s	CIF 30 frames/s
Search range	± 16	± 16
Block size	16×16 to 4×4	16×16 to 4×4
Min. required frequency (MHz)	24.1	16.04
Num. 16×8 -bit dual-port RAMs	129	144
Voltage (V)	1.5	2.5
Power consumed (mW)	25	25

H. Implementation Results and Analysis

The proposed architecture has been implemented in Verilog HDL. Simulation and functional verification of the architecture was done using ModelSim. We then synthesized the architecture using the Xilinx synthesis tool which is part of Xilinx integrated software environment (ISE 9.1). After synthesis, place and routing was done targeting the Virtex-II Pro XC2VP30 Xilinx FPGA on our development board. We obtain power analysis for our design, using the Xilinx XPower tool. We target QCIF video for our prototype.

Our implementation results are shown in Tables XVIII–XX. From Table XVIII we see that our architecture can achieve a maximum frequency of 246.5 MHz. The FPGA power consumption of our architecture is 25 mW and the total equivalent gate count is 268 K.

Our simulations in ModelSim show that it takes 27 cycles to obtain the minimum SAD from each search pass, after initialization. The 27 cycles are obtained from 1 cycle for the AGU, 1 cycle to read data from on-chip memory, 16 cycles for the PU, 5 cycles for the SAD combination tree, and 4 cycles for the comparison unit.

Therefore, it takes 270 (10×27) cycles to complete the search for 1 CMB, 1 reference frame and $s = \pm 16$. For a CIF image (396 MBs) at 30 Hz and considering 5 reference frames, a minimum clock frequency of approximately $16.04(270 \times 396 \times 30 \times 5)$ MHz is required.

Thus with a maximum possible clock speed of 246.5 MHz, our architecture can compute in real time CIF sequences within a search range of ± 16 and using five reference frames.

From Table XIX, we see that our architecture achieves the highest maximum frequency and throughput in frames/s, when compared to other FPGA-based architectures in [14]–[16], [21], and [23]. It also achieves the lowest estimated power consumption and the second lowest area. In Table XIX, pre-implementation power estimates are obtained using Xilinx power estimator, since post-implementation results are not available. Our post-implementation results are obtained using Xilinx XPower.

In Table XX, we compare our architecture with previous state-of-the-art architectures implemented on ASICs. Note that direct FPGA-ASIC comparison is impossible because of the

TABLE XIX
COMPARISON WITH OTHER FPSOC ARCHITECTURES

	Qiu <i>et al.</i> [21]	Canals <i>et al.</i> [14]	Rahman <i>et al.</i> [15]	Roma <i>et al.</i> [16]	Modified SUMH [23]	This Paper
FPSOC FPGA	Virtex-4	Virtex-4	Virtex-2	Virtex-E	Virtex-2 Pro	Virtex-2 Pro
Algorithm	ACQPPS	Full search	Full search	Full search	Modified SUMH	HMDS
Video format	QCIF	QCIF	QCIF	QCIF	QCIF	QCIF
Search range	±16	±48	±4	±16	±16	±16
Number of CLB slices	3.9 K	12.5 K	14.5 K	14.7 K	11.4 K	7.8 K
Number of LUTs	3.2 K	6.3 K	28.5 K	10.0 K	18.7 K	10.8 K
Clock frequency (MHz)	60	100	149.2	76.1	145.2	246.5
Gate count	280 K	429 K	225 K	505 K	388 K	268 K
Throughput (frames/s)	120	146	150	N/A	180	461
Est. pre-implementation power (mW)	551	841	965	1127	492	492
Post-implementation power (mW)	N/A	N/A	N/A	N/A	25	25

TABLE XX
COMPARISON WITH OTHER ARCHITECTURES IMPLEMENTED ON ASICs

	[17] Chao's	[18] Miyakoshi's	[19] Lin's	[20] Chen's	[23] Ours	Ours: This paper
Process (μm)	0.35	0.18	0.18	0.18	0.13 FPGA	0.13 FPGA
Voltage (V)	3.3	1.0	1.8	1.3	1.5	2.5
Transistors count	301 K	1000 K	546 K	708 K	388 K	268 K
Maximum frequency (MHz)	50	13.5	48.67	66	145.2	246.5
Vido spec.	CIF 30 frames/s	CIF 30 frames/s	CIF 30 frames/s	CIF 30 frames/s	CIF 30 frames/s	CIF 30 frames/s
Minimum frequency (MHz)	50	13.5	48.67	13.5	24.1	16.04
Algorithm	Diamond search	Gradient decent	4SS	Single-iteration parallel 4SS w/1-ref.	Modified SUMH	HMDS
Block size	16 × 16 and 8 × 8	16 × 16 and 8 × 8	16 × 16	16 × 16 to 4 × 4	16 × 16 to 4 × 4	16 × 16 to 4 × 4
Power (mW)	223.6	6.56	8.46	2.13	25	25
Normalized power (1.8 V, 0.18 μm) (*)	17.60	21.25	8.46	4.08	69.02	24.85
Architecture	1-D tree. No data reuse scheme.	1-D tree. No data reuse scheme.	1-D tree. Level A data reuse scheme.	2-D tree. Level B data reuse scheme.	1-D tree. No data reuse scheme.	1-D tree. No data reuse scheme.

$$(*) \text{ Normalized Power} = \text{Power} \times \frac{0.18^2}{\text{process}^2} \times \frac{1.8^2}{\text{voltage}^2}.$$

fact that the platforms are different. ASICs still achieve the highest performance in terms of area, power consumed and maximum frequency. However, we provide Table XX not for direct comparisons, but to show that our implementation achieves ASIC-like levels of performance. This is desirable because it indicates that an ASIC implementation of our architecture will yield even better performance results. Our Verilog implementation was kept portable in order to simplify FPGA to ASIC migration.

From Table XX we see that our architecture achieves many desirable results. The most remarkable is that the power consumption is very low despite the fact that our implementation is done on an FPGA which typically consumes more power than an ASIC. Besides the low power consumption of our architecture, other favorable results are that our architecture achieves the lowest transistor count. The algorithm we use (HMDS) also has better PSNR performance than the algorithms used in [17]–[20]. Finally, our architecture achieves comparably low minimum frequency which is attractive in low power applications.

V. CONCLUSION

In this paper, we presented a new hardware-oriented, integer-pel, fast ME algorithm called HMDS, along with its

supporting hardware architecture. We showed that at lower bit rates, HMDS has insignificant RD losses when compared to full search, while it has superior RD performance compared to some state-of-the-art fast ME algorithms. HMDS superior RD performance and its suitability for hardware implementation, arise from the fact that with a fixed and regular search pattern, HMDS is robust enough to find the globally optimum motion vector because it simultaneously tracks large and local motions through its satellite search points and last two steps of intensive local search. We showed that HMDS has superior speedup than many fast algorithms, while its loss in speedup is minimal when compared to SUMH.

We showed that our low power supporting architecture achieves better hardware efficiency than some FPGA-based architectures, and its performance is comparable to ASIC-implemented architectures.

For low bit rate applications and low power mobile devices, which typically use H.264/AVC baseline profile at levels 1–2, HMDS is more attractive than the advanced fast ME algorithms. This is because it is hardware-oriented and thus is more amenable to efficient hardware acceleration. HMDS is also more attractive than full search and the modified SUMH because while the latter two algorithms are hardware-oriented, they are more complex than HMDS and thus less hardware-efficient. Thus for low bit rate applications and low power

mobile devices, HMDS can be used without much penalty, instead of full search, for ME in H.264/AVC.

ACKNOWLEDGMENT

The authors acknowledge the support from Xilinx, Inc., San Jose, CA, the Xilinx University Program, the Packard Foundation, Los Altos, CA, and the Department of Electrical Engineering, Santa Clara University, Santa Clara, CA. They would also like to thank the editors and reviewers of this journal for their useful comments.

REFERENCES

- [1] T. Wiegand, G. J. Sullivan, G. Bjøntegaard, and A. Luthra, "Overview of the H.264/AVC video coding standard," *IEEE Trans. Circuits Syst. Video Technol.*, vol. 13, no. 7, pp. 560–576, Jul. 2003.
- [2] T. Koga, K. Iinuma, A. Hirano, Y. Iijima, and T. Ishiguro, "Motion compensated interframe coding for video conferencing," in *Proc. Nat. Telecommun. Conf.*, Nov. 1981, pp. G5.3.1–G5.3.5.
- [3] R. Li, B. Zeng, and M. L. Liou, "A new three-step search algorithm for block motion estimation," *IEEE Trans. Circuits Syst. Video Technol.*, vol. 4, no. 4, pp. 438–482, Aug. 1994.
- [4] L.-M. Po and W.-C. Ma, "A novel four-step search algorithm for fast block motion estimation," *IEEE Trans. Circuits Syst. Video Technol.*, vol. 6, no. 3, pp. 313–317, Jun. 1996.
- [5] J. Y. Tham, S. Ranganath, M. Ranganath, and A. A. Kassim, "A novel unrestricted center-biased diamond search algorithm for block motion estimation," *IEEE Trans. Circuits Syst. Video Technol.*, vol. 8, no. 4, pp. 369–377, Aug. 1998.
- [6] S. Zhu and K. K. Ma, "A new diamond search algorithm for fast block-matching motion estimation," *IEEE Trans. Image Process.*, vol. 9, no. 2, pp. 287–290, Feb. 2000.
- [7] C. Zhu, X. Lin, and L. P. Chau, "Hexagon-based search pattern for fast block motion estimation," *IEEE Trans. Circuits Syst. Video Technol.*, vol. 12, no. 5, pp. 349–355, May 2002.
- [8] *Coding of Moving Pictures and Audio*, ISO/IEC JTC1/SC29/WG11 N2932, Oct. 1999.
- [9] A. M. Tourapis, "Enhanced predictive zonal search for single and multiple frame motion estimation," *Proc. SPIE Vis. Commun. Image Process.*, vol. 4671, pp. 1069–1079, Jan. 2002.
- [10] C. H. Cheung and L. M. Po, "A novel cross-diamond search algorithm for fast block motion estimation," *IEEE Trans. Circuits Syst. Video Technol.*, vol. 12, no. 12, pp. 1168–1177, Dec. 2002.
- [11] C. H. Cheung and L. M. Po, "Novel cross-diamond-hexagonal search algorithms for fast block motion estimation," *IEEE Trans. Multimedia*, vol. 7, no. 1, pp. 16–22, Feb. 2005.
- [12] Z. Chen, P. Zhou, and Y. He, *Fast Integer PEL and Fractional PEL Motion Estimation for JVT*, document JVT-F017, Joint Video Team (JVT) of ISO/IEC MPEG and ITU-T VCE, 6th Meeting, Awaji Island, Japan, Dec. 2002.
- [13] X. Yi, J. Zhang, N. Ling, and W. Shang, *Improved and Simplified Fast Motion Estimation for JM*, document JVT-P021.doc, Joint Video Team (JVT) of ISO/IEC MPEG and ITU-T VCEG, 16th Meeting, Posnan, Poland, Jul. 2005.
- [14] J. A. Canals, M. A. Martínez, F. J. Ballester, and A. Mora, "New FPSoC-based architecture for efficient FSBM motion estimation processing in video standards," *Proc. SPIE*, vol. 6590, p. 65901N, May 2007.
- [15] C. Rahman and W. Badawy, "A quarter PEL full search block motion estimation architecture for H.264/AVC," in *Proc. IEEE ICME*, Jul. 2005, pp. 414–417.
- [16] N. Roma, T. Dias, and L. Sousa, "Customizable core-based architectures for real-time motion estimation on FPGAs," in *Proc. 3rd Int. Conf. Field Prog. Logic Appl.*, Sep. 2003, pp. 745–754.
- [17] W.-M. Chao, C.-W. Hsu, Y.-C. Chang, and L.-G. Chen, "A novel hybrid motion estimator supporting diamond search and fast full search," in *Proc. IEEE Int. Symp. Circuits Syst.*, vol. 2, May 2002, pp. II-492–II-495.
- [18] J. Miyakoshi, Y. Kuroda, M. Miyama, K. Imamura, H. Hashimoto, and M. Yoshimoto, "A sub-mW MPEG-4 motion estimation processor core for mobile video application," in *Proc. IEEE Custom Integr. Circuits Conf.*, Sep. 2003, pp. 181–184.
- [19] S.-S. Lin, "Low-power motion estimation processors for mobile video application," M.S. thesis, Graduate Inst. Electron. Eng., Nat. Taiwan Univ., Taipei, Taiwan, 2004.
- [20] T.-C. Chen, Y.-H. Chen, S.-F. Tsai, S.-Y. Chien, and L.-G. Chen, "Fast algorithm and architecture design of low-power integer motion estimation for H.264/AVC," *IEEE Trans. Circuits Syst. Video Technol.*, vol. 17, no. 5, pp. 568–577, May 2007.
- [21] Y. Qiu and W. Badawy, "A prototyping virtual socket system-on-platform architecture with a novel ACQPPS motion estimator for H.264 video encoding applications," *EURASIP J. Embedded Syst.*, vol. 2009, no. 105979, pp. 1–20, 2009.
- [22] O. Ndili and T. Ogunfunmi, "A hardware oriented integer pel fast motion estimation algorithm in H.264/AVC," in *Proc. IEEE Conf. DASIP*, Nov. 2008, pp. 314–320.
- [23] O. Ndili and T. Ogunfunmi, "FPSoC-based architecture for a fast motion estimation algorithm in H.264/AVC," *EURASIP J. Embedded Syst.*, vol. 2009, no. 893897, pp. 1–16, 2009.
- [24] O. Ndili and T. Ogunfunmi, "Hardware-oriented modified diamond search for motion estimation in H.264/AVC," in *Proc. IEEE ICIP*, Sep. 2010, pp. 749–752.
- [25] *H.264/AVC Reference Software JM 13.2*. (2009) [Online]. Available: <http://iphome.hhi.de/suehring/tml/download>
- [26] C.-Y. Chen, C.-Y. Huang, Y.-H. Chen, and L.-G. Chen, "Level C+ data reuse scheme for motion estimation with corresponding coding orders," *IEEE Trans. Circuits Syst. Video Technol.*, vol. 16, no. 4, pp. 553–558, Apr. 2006.



Obianuju Ndili received the B.S. degree in electrical engineering from the University of Lagos, Lagos, Nigeria, in 1998, the M.S. degree in electrical and computer engineering from Cornell University, Ithaca, NY, in 2003, and the Ph.D. degree in electrical engineering from Santa Clara University, Santa Clara, CA, in 2011.

She is currently with Google, Inc., Mountain View, CA. Her current research interests include applied signal processing and information theory, video compression, very large scale integration design, and embedded systems. She has over a dozen refereed publications in these areas.



Tokunbo Ogunfunmi (SM'00) received the B.S. (with first class honors) degree from the University of Ife, Ife, Nigeria, and the M.S. and Ph.D. degrees from Stanford University, Stanford, CA, all in electrical engineering.

He is the Associate Dean for Research and Faculty Development with the School of Engineering, an Associate Professor with the Department of Electrical Engineering, and the Director of the Signal Processing Research Laboratory, Santa Clara University (SCU), Santa Clara, CA. In 2003, he served as the Acting Chair of the Department of Electrical Engineering. At SCU, he teaches a variety of courses in signal processing and related areas. His current research interests include digital and adaptive signal processing and applications, nonlinear signal processing, and speech and multimedia (audio, video) compression. He has published two books and over 140 refereed journal and conference papers in these areas. He has been a consultant to industry and government, and was a Visiting Professor with the University of Texas, Austin, and with Stanford University. He has consulted for companies such as Broadcom, Irvine, CA, AMD, Sunnyvale, CA, CLARIS Corporation, Santa Clara, AT&T Bell Laboratories, Murray Hill, NJ, and NIKON Precision Research and Development, Belmont, CA.

Dr. Ogunfunmi is an Associate Editor of the *Journal of Circuits, Systems and Signal Processing*. He has served on many IEEE conference committees as a member of the organizing and technical committees. He has also organized several special sessions in these conferences. He is a member of Sigma Xi (the Scientific Research Society) and the American Association for the Advancement of Science. He served as the Vice-Chair and the Chair of the IEEE Signal Processing Society Santa Clara Valley Chapter. He is a member of the IEEE Circuits and Systems (CAS) Society technical committees on multimedia, digital signal processing, circuits and systems for communications, and circuits and systems for education outreach. He is a Registered Professional Engineer in California.